

13. Desktop Applications with PyWebView

Detailed beginner handbook - English and Arabic

Technologies: PyWebView

What you will learn

Open a local web page in a native desktop window.

PyWebView wraps local web content in a native window and can provide a careful Python-to-JavaScript bridge.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولاً، ثم جرب المثال الصغير بيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/13-pywebview.
3. Read README.md.
4. Run `pip install -r requirements.txt; python app.py`.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat PyWebView as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - How a native window can host a local web application

What is it?

PyWebView creates a native window containing the platform web engine. The interface remains HTML/CSS/JavaScript while Python manages the process and native integration.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
import webview
webview.create_window('Learning app', 'index.html', width=900, height=650)
webview.start()
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates How a native window can host a local web application.

How a native window can host a local web application: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Creating windows, choosing a URL, sizing, and lifecycle

What is it?

`create_window` defines title, URL, width, height, and options. `webview.start` enters the GUI event loop, so initialise required services before or through a coordinated callback.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
window=webview.create_window('Dashboard', 'http://127.0.0.1:8000')
# webview.start() enters the GUI event loop.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Creating windows, choosing a URL, sizing, and lifecycle.

Creating windows, choosing a URL, sizing, and lifecycle: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - Starting a local API before opening the window

What is it?

A desktop web app often starts Uvicorn on localhost in a background thread, waits for a health endpoint, then points the window at the built frontend URL.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
threading.Thread(target=server.run,daemon=True).start()  
wait_until_ready('http://127.0.0.1:8000/health')  
webview.create_window('App', 'http://127.0.0.1:8000')
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Starting a local API before opening the window.

Starting a local API before opening the window. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح: window

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - Choosing a free localhost port and waiting for readiness

What is it?

Bind a temporary socket to port zero to obtain a free port, start the server, and poll readiness with a timeout. Report a clear error if startup fails.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
with socket.socket() as sock:
    sock.bind(('127.0.0.1',0))
    port=sock.getsockname()[1]
print(port)
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Choosing a free localhost port and waiting for readiness.

Choosing a free localhost port and waiting for readiness: اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - The JavaScript-Python bridge and strict API boundaries

What is it?

js_api exposes selected Python methods to JavaScript. Treat every call as untrusted input, expose the smallest surface, validate parameters, and avoid arbitrary filesystem access.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
class Api:
    def choose_month(self, month):
        if month not in {'Jan', 'Feb'}: raise ValueError('Invalid month')
        return {'month': month}
webview.create_window('App', 'index.html', js_api=Api())
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates The JavaScript-Python bridge and strict API boundaries.

The JavaScript-Python bridge and strict API boundaries: اكتب المثال بنفسك، حدد المدخلات والعمليات والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Native open/save/folder dialogs

What is it?

Native dialogs can select files and folders. Constrain extensions where appropriate, handle cancellation, normalise returned paths, and never assume the user selected a valid workbook.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
result=webview.windows[0].create_file_dialog(  
    webview.FileDialog.OPEN, '', False, '', ('Excel (*.xlsx)',))  
if not result: print('Cancelled')
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Native open/save/folder dialogs.

يشرح: Native open/save/folder dialogs. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.
هذا الدرس

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Downloads, external links, permissions, and WebView2

What is it?

Allow downloads deliberately and open external links in the default browser rather than the trusted app window. WebView2 is the normal Windows rendering engine.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
webview.settings['ALLOW_DOWNLOADS']=True
webview.settings['OPEN_EXTERNAL_LINKS_IN_BROWSER']=True
# Validate every selected path.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Downloads, external links, permissions, and WebView2.

Downloads, external links, permissions, and WebView2: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Threads, shutdown, cleanup, and preventing orphan servers

What is it?

Coordinate server and window shutdown, signal Uvicorn to exit, join threads with limits, and clean temporary files so closing the app does not leave processes behind.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
try:
    webview.start()
finally:
    server.should_exit=True
    thread.join(timeout=5)
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Threads, shutdown, cleanup, and preventing orphan servers.

Threads, shutdown, cleanup, and preventing orphan servers: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Development browser mode versus packaged desktop mode

What is it?

During development, a browser and Vite server provide better debugging. In packaged mode, serve built files and open the same application inside PyWebView.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# Development: Vite browser URL for DevTools.  
# Package: built frontend served locally inside Edge WebView2.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Development browser mode versus packaged desktop mode.

Development browser mode versus packaged desktop mode: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Security, debugging, logs, and platform differences

What is it?

Bind to localhost, randomise ports, validate bridge calls, log startup stages, hide sensitive values, and test WebView2 availability and file dialogs on supported Windows versions.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Desktop Applications with PyWebView, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# Bind only to 127.0.0.1. Validate bridge inputs.  
# Log startup stages without data values. Check WebView2 before launch.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Security, debugging, logs, and platform differences.

Security, debugging, logs, and platform differences: اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Open a local web page in a native desktop window.

Starting example

```
import webview
webview.create_window('Learning demo', 'index.html')
webview.start()
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines PyWebView with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة أمانة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step

Next: 14. Windows Packaging